SCALE FOR PROJECT PYTHON MODULE (/PROJECTS/PYTHON-MODULE-05)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42heilbronn.de:vogsphere/intra-uuid-86929f74-c7de
```

Introduction

- Remain polite, courteous, respectful, and constructive
throughout the evaluation process. The well-being of the community
depends on it.

- Together with the person (or group) being evaluated, identify any issues in the work.
Take the time to discuss and debate the problems you have identified.

- You must consider that there might be differences in how your
peers have understood the project's instructions and the
scope of its functionality. Always keep an open mind and grade
their work as honestly as possible. The pedagogy is valid
only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's
Git repository.

- Double-check that the Git repository belongs to the student
or the group. Ensure that the work is for the relevant project,
and also check that "git clone" command is run in an empty folder.

- Check carefully that no malicious aliases were used to fool you
and make you evaluate something other than the content of the
official repository.

- To avoid any surprises, carefully check that both the evaluating
and the evaluated students have reviewed any scripts that may be used
to facilitate grading.

- If the evaluating student has not completed that particular
project yet, it is mandatory for this student to read the
entire subject document prior to starting the defence.

- Use the flags available on this scale to signal an empty repository,
a non-functioning program, a norm error, cheating, etc. In these cases,
the grading is over and the final grade is 0 (or -42 in the case of
cheating). However, with the exception of cheating, you are
encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no unexpected,
premature, uncontrolled termination of the program is allowed;
otherwise, the final grade is 0. Use the appropriate flag.
You should never have to edit any file other than the configuration file, if one exists.
If you want to edit a file, take the time to discuss the reasons with the
evaluated student and make sure both of you agree with this change.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/203189/en.subject.pdf)

Preliminaries

Basics

- Only grade the work that is in the learner's Git repository
- Ensure the Git repository belongs to the learner
- Check that no malicious aliases are used
- Verify the project structure matches the subject requirements
- Check that only the requested files are available in the Git repository. If not, the evaluation st

Are all preliminary checks satisfied?

Yes No

Exercises

Exercise 0 - Data Processor

If needed, ask the assessed person to help you with the tests.
Does the program correctly implement ALL the following features?

- A file named data_processor.py exists
- The program imports the abc module for abstract base classes, or specific elements from it (
- DataProcessor is an abstract base class (inherits from ABC)
- DataProcessor has @abstractmethod decorators on the validate() and ingest() methods
- Method signatures match requirements:
  - validate(self, data: Any) -> bool
  - ingest(self, data: Any) -> None
  - output(self) -> tuple[int, str]
- The NumericProcessor(), TextProcessor(), and LogProcessor() classes inherit from DataProc
- All subclasses properly override the abstract methods with specialized behavior
- NumericProcessor correctly validates and ingests numeric data (int, float, and lists of both typ
- TextProcessor correctly validates and ingests string data (str and list of str)
- LogProcessor correctly validates and ingests log data as a dict of strings (one per log entry) a (multiple log entries)
- Run python3 data_processor.py and verify that multiple tests cover the expected behaviours: functions can detect valid and invalid data for each specialised class, the data is correctly ing transformed into strings, and each call to output() on a data processor returns the oldest elen removes it.
- The program tests an ingest() with invalid data, and it raises an error correctly caught (this im error)
- Ask the assessed person to explain why abstract base classes are useful

Yes No

Exercise 1 - Polymorphic Processing of Data Streams

If needed, ask the assessed person to help you with the tests.
Does the program correctly implement ALL the following features?

- A file named data_stream.py exists
- The program is an improvement on the previous exercise, and all previously verified behavio present
- DataStream is a new class that implements register_processor() and process_stream()
- Method signatures match the following requirements:
  - register_processor(self, proc: DataProcessor) -> None
  - process_stream(self, stream: list[typing.Any]) -> None
  - print_processors_stats(self) -> None
- The process_stream() method calls the validate() and ingest() methods from all registered da using polymorphic calls
- The program correctly handles invalid data in the stream
- The statistics are displayed for all registered data processors, either by accessing attributes an extra method
  from each data processor
- Ask the assessed person to explain how polymorphism enables the DataProcessor class to h data types in a stream

Yes No

Exercise 2 - Data Pipeline

If needed, ask the assessed person to help you with the tests.
Does the program correctly implement ALL the following features?

- A file named data_pipeline.py exists
- The program is based on the previous exercise and extends it
- The new class ExportPlugin is created and inherits from Protocol
- The ExportPlugin class defines a method with the following signature: process_output(self, d str]]) -> None
- At least two independent classes (CSV and JSON) exist and conform to the ExportPlugin coı
- The DataStream class has a new method that exports the processed data from the stream tc output_pipeline(self, nb: int, plugin: ExportPlugin) -> None
- Ask the learner to explain the difference between Protocol (duck typing) and ABC (abstract b

Yes No

Code Quality and Understanding

Code Quality and Understanding

Evaluate the overall code quality and learner understanding:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Comprehensive type annotations are used throughout (all functions have parameter and retu this using mypy (note that there is an exception on Ex0).
- Docstrings are NOT required for this module
- Proper use of ABC and @abstractmethod decorators for abstract base classes
- Proper use of Protocol for plugins
- Error handling is implemented appropriately
- The learner can explain what abstract base classes (ABC) are and why they're useful
- The learner understands when and why to use polymorphism
- The learner can demonstrate how the same interface works with different implementations
- The learner understands that @abstractmethod forces subclasses to implement specific met

Yes No

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

Concerning situation Leaks Forbidden function Can’t support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation